

Features:

The system uses the following specifications to meet the individualised parameters:

Bus Speed: 14MHz

Operating Voltage: 3.3V

Cost: \$14 for the ToF sensor. \$7 for the stepper motor and driver. Tape, cardboard, and wires - about \$10. Total cost apart from the MCU = \$31.

Serial Communication: UART protocol between the ARM MCU and PC at a baud rate of 115,200.

Programming Language: C (Microcontroller - For functionality and setup) and MATLAB (Receiver - PC - For visualisation)

Memory: Our microcontroller uses 1024 KB of Flash memory. It also uses 256KB of single-cycle SRAM. It uses 6KB of EEPROM. [1].

General Description:

The system performs 3 Dimensional spatial mapping. It uses a specific structure of acquiring signals. A Time of Flight (ToF) sensor is mounted on a rotating stepper motor.

Signal Acquisition;

The VL53L1X Time of Flight (ToF) sensor emits light pulses at a frequency of 940nm. It emits a class1 invisible laser. The SPAD (Single Photon Avalanche Diode) is the receiving array which catches the reflected light. [2]. It uses the general relationship between velocity and time to find the distance.

Pre-Processing:

$v = d/t$, $c = 2d/t$, where c is the speed of light (approx $3e8\text{m/s}$). t is the time it takes for the light to come back to the SPAD, and d is the distance travelled. A multiplier of 2 is used to indicate that the light travels the same distance twice due to reflection. The ToF sensor uses digital signal processing to find this distance

then provides the data to a master device using I2C communication. It finds the time taken for the reflected light to come back. Then, it converts the data into distance (mm). This is where the analog data is converted to digital data. It shows how an intelligent system can see the outside world and convert its environment's data into a readable format.

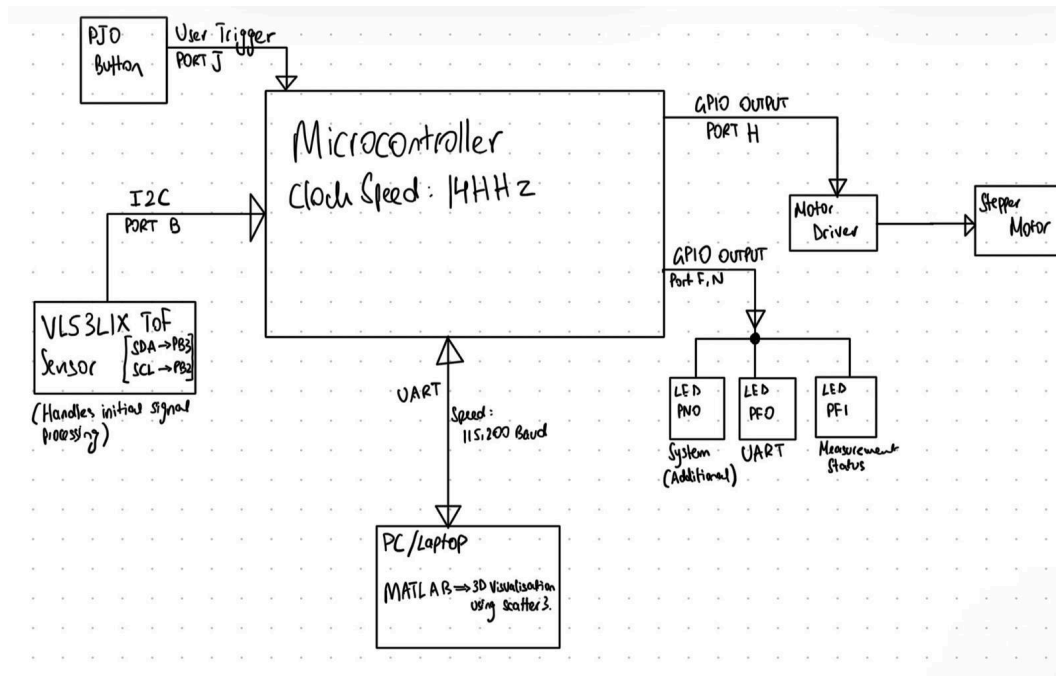
Communication:

The MCU acts as an I2C master to the ToF sensor and communicates using the SDA (Data) and SCL (Clock) lines on the sensor and MCU. The ToF sensor and MCU communicate with the PC using UART communication. The Baud rate for UART was 115,200. It is used to transmit the distance and angle measurements to the PC. This data was then used in MATLAB for further clarification.

Visualization:

MATLAB receives the data packets from the ToF sensor. The sensor does the distance calculations itself. However, we need the x,y,z axis mapping using MATLAB. It performs trigonometric calculations on distance values using the motor angle and creates a 3D map.

Block Diagram:



Device Characteristics Table:

Characteristics	Specification
Pins Used	ToF (I2C): SDA-PB3, SCL-PB2 Stepper Motor: Port H, PH0 to PH3 Button: PJ0 Onboard LEDs: Measurement Status: PF4 UART Tx: PF0 Additional Status: PN0 UART: PA0 and PA1 on the USB channel
Bus Speed	14MHz
Communication Speed	UART: 115,200 Baud

Detailed Description:

Distance Measurement Acquisition:

The VL53L1X ToF sensor uses light and physics to find the distance of the nearest object. It makes use of two films attached in the front. One is used to emit light and the other, SPAD (Single Photon Avalanche Diode), is used to receive the reflected light pulse. The sensor can take a maximum measurement of 4m. It uses the principles of speed, distance, and time. The formula can be seen below

$v = d/t$, $c = 2d/t$, where c is the speed of light, d is the distance to be calculated, and t is the time taken for the light photon to travel

The MCU communicates via I2C to trigger ranging and receive results in millimeters (mm). The SDA line of the ToF sensor is connected to pin PB3 of the MCU. The SCL line is connected to pin PB2 of the MCU. For this project, 32 measurements were taken per 360 degree rotation to ensure a high-resolution and packed mapping. The sensor is event driven which means that every time a measurement is taken, the motor stops for a very small amount of time. The process of data signal acquisition occurs in the interrupt service routine. This is to ensure the sensor accurately measures the distance.

Distance Measurement:

The MCU sends in raw polar coordinates to the PC which need to be converted to a 3D cartesian coordinate system. The conversion was carried out on the MATLAB side to ensure the MCU is not spending too much power doing calculations and dealing with floating point errors. Furthermore, the bus speed is 14MHz which means our clock is restricted.

Formulas Used:

$$z = d * \cos(\Theta)$$

$$y = d * \sin(\Theta)$$

$$x = \text{Displacement (0mm, 100mm, 200mm)}$$

Example Calculation:

ToF Distance measurement = 1000mm

Stepper Motor Angle = 45 degrees

x = 10mm

$$z = 1000 * \cos(45) = 707.1mm$$

$$y = 1000 * \sin(45) = 707.1mm$$

Coordinates plotted on the 3D map: (100,707.1,707.1)

Displacement Implementation:

The x-axis displacement is implemented through a manual ‘simulated’ displacement. After each 360 degree turn, the motor turns back to the initial position during which the x value is incremented by 100mm. For the hallway scans, the system was manually moved 10cm forward which reflected the ‘simulated’ x value as well. This allows us to reconstruct the 3D hallway scans.

Visualisation Software:

The visualisation is handled by MATLAB on the PC side. The data is being read from the PC’s communication port which is connected to the MCU and communicating through UART. The data is being read line-by-line to ensure all distance and angle measurements are read thoroughly.

The process involves getting all the points then plotting them at once in a 3 dimensional space. The function used for this is `scatter3(X,Y,Z)`. It displays circles at the locations specified by x,y,z as vectors.

Functions used for communication are `configureTerminator(device,terminator)` and `serialport("COM5", 115200);` `configureTerminator` allows read/write communication with the specified serial port.

The `readline(line)` function is used to read every line from the data being received.

Another function being used is the `str2num(txt)` function. This converts a character array to a numeric matrix. If it is unable to parse something, it just returns an empty matrix, not allowing errors to throw the entire program flow into panic.

The function `deg2rad(D)` is used to convert the stepper angle in degrees to radians for the sine and cosine calculations.

Application Example:

Figure 1 shows the system in a hallway which was spatially mapped using this system. Figure 2 shows the results of the mapping in MATLAB. A detailed image of the setup can be seen in figure 3.

In order to achieve these results, we can attach the ToF sensor to the stepper motor shaft using a 3D printed mount. Then, we can wire all the components to the MCU while ensuring there is enough space for all the components. Once the system is set up, we need to translate, build, and flash the code to the MCU. The ToF sensor has been attached to the setup in a protruding way to ensure a full 360 degree scan is covered. Pick up the setup at about chest height and carry the PC on the side so the MCU can remain powered on and MATLAB can keep communicating and receiving data from the sensor. Then, on MATLAB, run the script responsible for gathering the data. Once everything is ready, press the reset button on the MCU and then press J0 (onboard button) to start the system.

The motor will start rotating, while taking very small stops in between for the ToF sensor to gather data and send readings to the PC. The system takes a total of 32 measurements using motor angle increments of 11.25 degrees. Stand still until that is done. Then, while the motor returns to home position without scanning, we take a step forward (approximately 10cm) to give a horizontal displacement for our 3D mapping. Do this until the end of the hallway, and in the corridor as well. A total of 20 scans were done in this case to get a good overview of the space.

The y-axis is the vertical axis in reference to the system. This can be seen in the graphs, where the central position of the setup is the middle. In reference to that, we see any points below it being in the negative y axis and anything above in the positive y axis. The z-axis takes mappings of the horizontal distance from the setup. It also goes from negative to positive, indicating our reference to be at the setup. The x-axis moves in a different plane. It is the displacement which is always positive in our case. This implementation can be seen in the calculations done for finding these distance measurements and mapping.

The figures of the 3D spatial mapping can be explained using the image of the hallway. Starting off, it scans a good view of a rectangular box. Any small dots seen closer to the reference point than the walls of the room are the wires connected to the ToF sensor. They are also picked up during the scanning. However, due to a small amount of readings and anomalies, the wiring does not hurt the overall spatial mapping. Around the end, the setup has also captured the bins placed to the left. This shows the small cluster of points near the corridor. Once we get into the corridor, we can see that the rectangular box made by the setup is smaller in height and width. This indicates that the system has the awareness and functionality to identify sudden changes in the environment. The points on the y-axis dip down, indicating the area got smaller.



Figure 1. System placed in the hallway it is supposed to scan

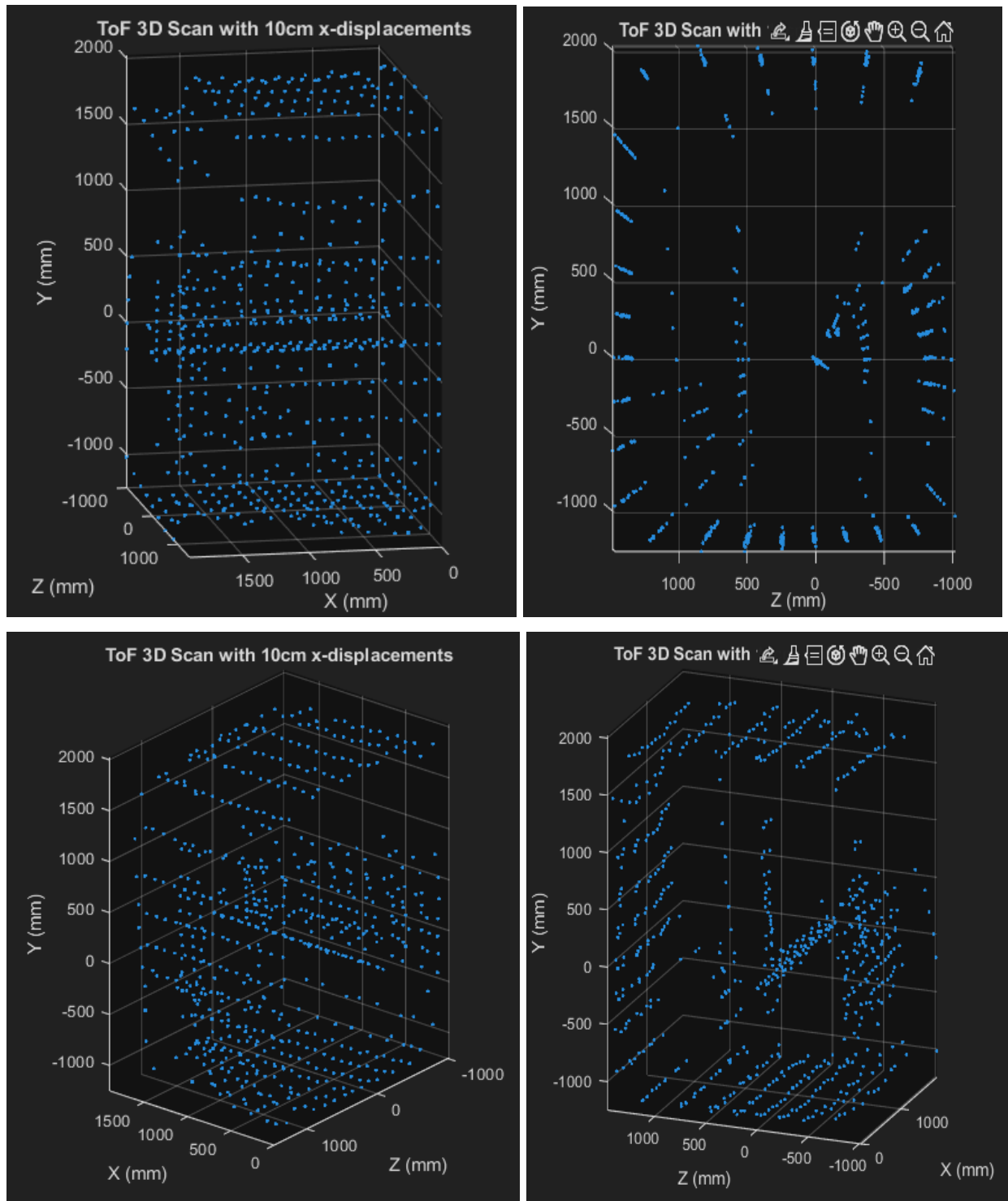


Figure 2. Spatial Mapping with different viewing angles



Figure 3. Image of the system used for scanning

Limitations:

FPU and Trigonometric Limitations:

The MCU has a hardware Floating Point Unit (FPU). However, at a low bus speed of 14MHz, calculating the trigonometric functions after every measurement in real-time can cause latency and potential failures in the processing. Furthermore, a C library, `<math.h>` is used for the trigonometric calculations which can also introduce rounding errors. Together, these limitations do not allow the y and z values to be calculated on the MCU. It has the ability to slow down the motor, delay sensor readings, and potentially cause a mechanical fault in the cardboard setup from the stepper motor movements. Due to these reasons, the y and z distance was instead calculated in the visualisation stage. As soon as a line of data was received from the sensor using UART, the MATLAB code converted it into actual cartesian distance coordinates by using the built-in math functions. This reduces the load on the MCU and allows it to operate well on this low bus speed. Another thing is that FPU is single-precision which is a hardware limitation.

Maximum Standard Serial Communication Rate:

The maximum communication rate for UART used in our case is 115,200 Baud. This can be picked from our lectures and design studios [3]. On realTerm, we need a baud rate of 115,200 in order for the MCU to communicate with the PC. Anything lower produces garbage values. The maximum baud rate my PC can handle is 128,000. This can be verified by going into device manager, clicking on the UART port connection, clicking on properties, and then scrolling to check the maximum setting for baud rate. Our system uses 115,200 which is smaller than the maximum value.

Communication Methods and Speeds:

From the MCU to ToF sensor, I2C communication is used. It is being used at the standard mode. This can be verified from the final project ToF sensor API code does not set any bits in order to increase the speed of the communication process. Standard mode will operate at 100kbps. However, due to changes in clock speed, the actual speed might differ.

```
I2C0_MTPR_R = 0b00000000000000101000000000111011;
```

The line of code above comes from the final project code during the initialisation part of the I2C communication line. This sets the default speed of 100kbps. This is the I2C Master Timer Period. It sets the clock rate for SCL. The lower 7 bits are used to set the I2C clock speed. [4]. For this line of code, the value of the lower 7 bits is 59 (0111011=(59)₁₀). This has been set up for our MCU's basic clock speed of 120MHz. Since the new clock speed is set up as 14MHz, this means that the sensor is not communicating at 100kbps.

To set the I2C clock speed, we need to use the following formula:

$$SCLperiod = 20 \times (1 + TPR) / SystemClockFrequency$$

$$Period = 20 \times (60) / 14M = 0.00008571428, \text{ frequency} = 1/period = 11.7kHz$$

$$Speed = 11.7kbps$$

Despite being slower, there is no indication in the setup regarding delays and latency due to the ToF sensor sending data slowly. There is definitely a difference in operating speed, but it is not noticeable due to the ToF running on interrupts and dominating when gathering data. The I2C was running at 11.7kbps.

System Bottlenecks:

Reviewing the entire system, the primary limitation on scanning speed can come from both the stepper motor and smaller clock frequency. A smaller frequency means that 1 clock cycle would take more time to finish. Therefore, it is more time consuming and can therefore slow down other components. The clock speed was tested using a GPIO pin on the MCU which shot out a HIGH and a LOW at two delays of 1400 clock cycles using SysTick_wait(clock_cycles). The waveform on the AD3 gave out a frequency of 4.8kHz which is really close to the expected 4.996kHz.

$$1 \text{ clock cycle} = 1 / 14e6 = 71.43ns$$

$$\text{systickwait}(1400) \Rightarrow 1401 \text{ cycles} \times 71.43n = 100.07us \text{ per half}$$

$$\text{Full period} = 200.14us$$

Expected frequency = 4.996kHz

The most significant bottleneck is the stepper motor. It needs magnetism and physical inertia to operate efficiently. For this, it requires delays alongside binary manipulation. The added delay is about 8400 clock cycles which produce a delay of 1.2ms for a whole binary iteration of the motor. This reduces overall speed. However, this was not noticeable in the system. This can be verified by observing motor failure when we reduce the SysTick delay. The motor either stops working at all, or the motor driver shows all lights as ON forever. Sometimes, there are noises as well.

Circuit Schematic:

The image of the board and its connected components can be seen below in figure 4.

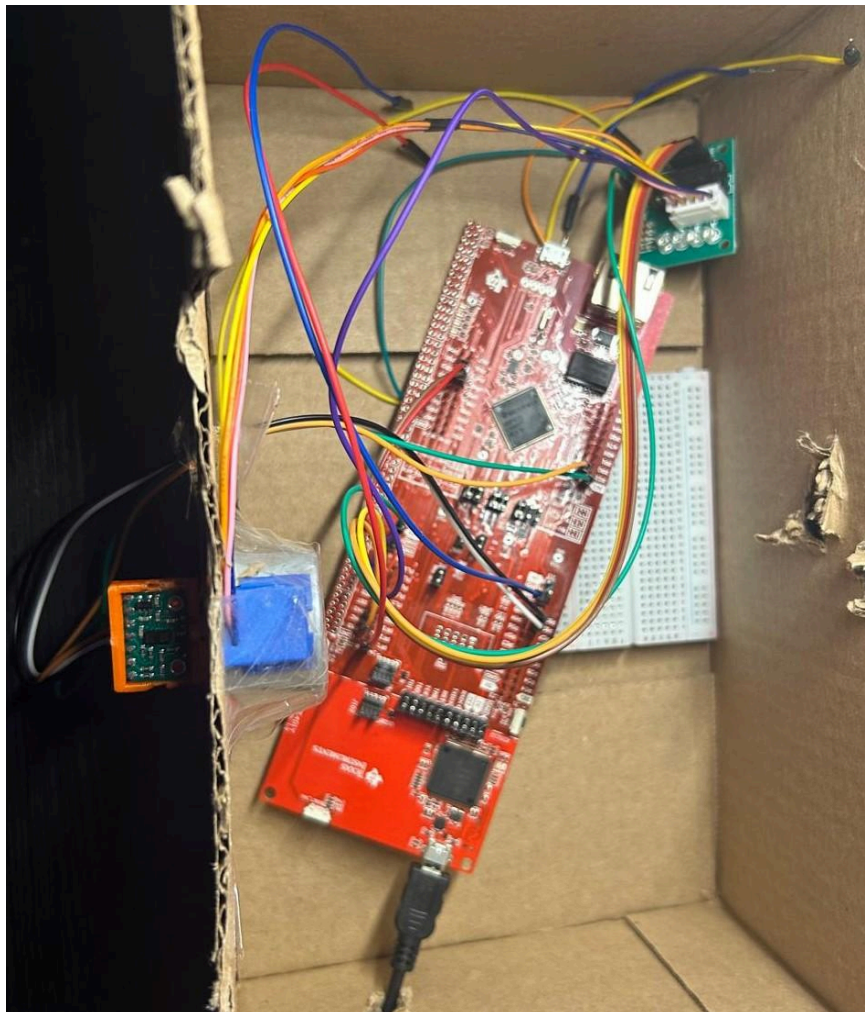


Figure 4. Close-up Image of the system used for scanning
A general schematic can be seen below in figure 5.

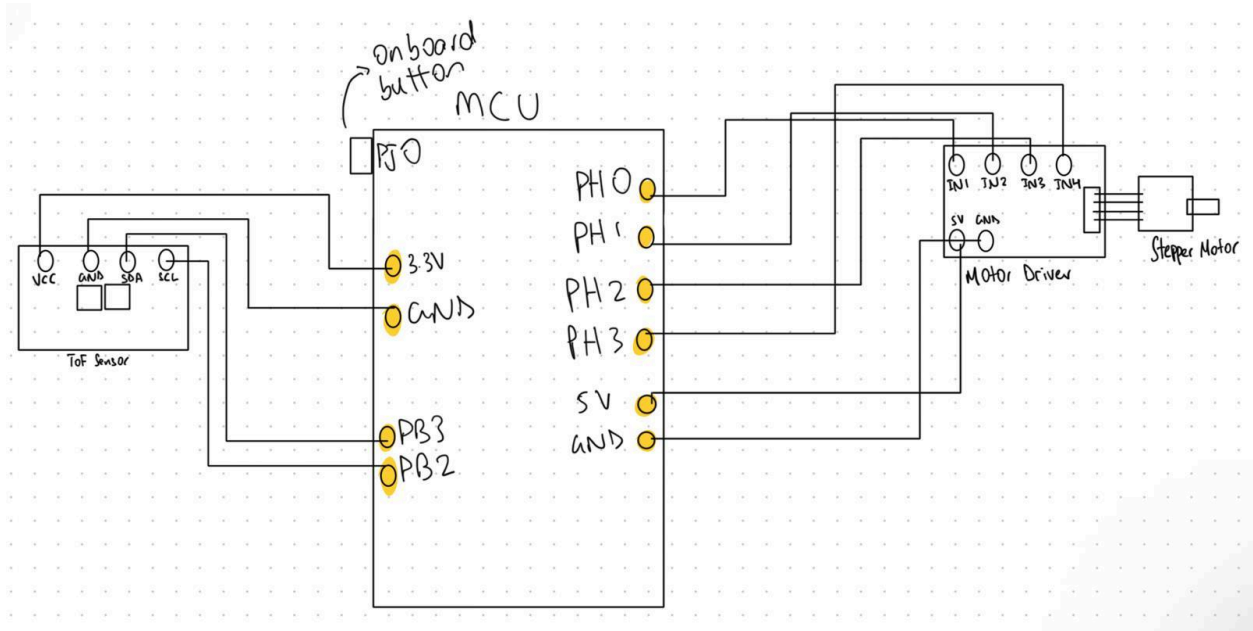
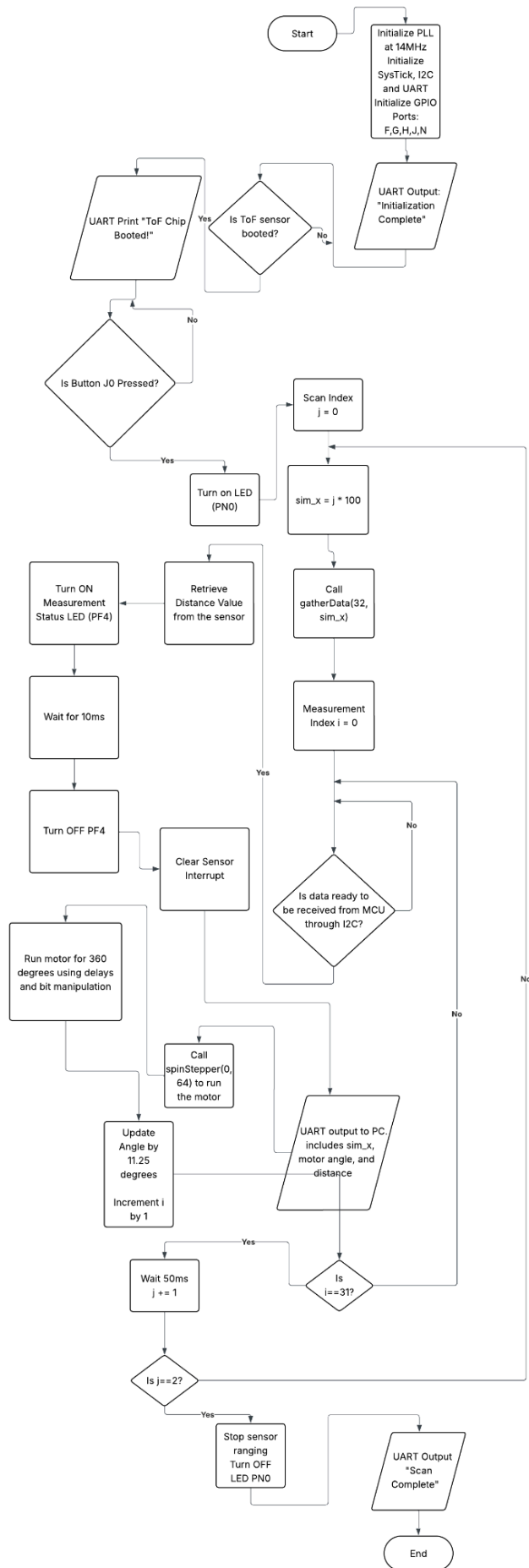
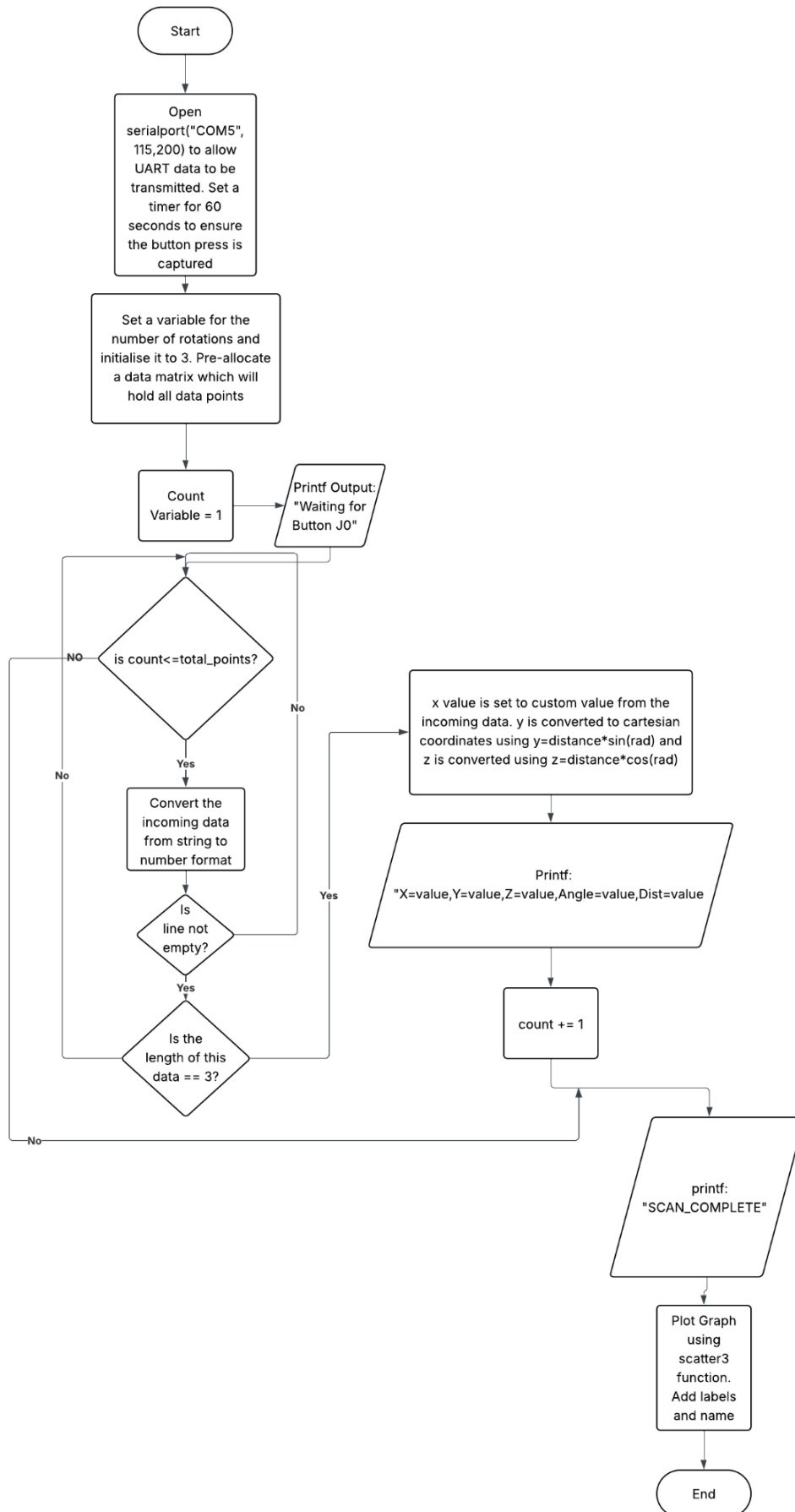


Figure 5. Circuit Schematic of the setup. The pins have been clearly outlined and connected.

Programming Logic Flowchart:



MCU SIDE



PC SIDE

References

- [1]. MSP432E401Y SimpleLink™ Ethernet Microcontroller datasheet.
<https://www.ti.com/lit/ds/symLink/mSP432e401y.pdf?ts=1775188835716> (accessed April. 03, 2026)
- [2]. Datasheet - VL53L1X - A new generation, long distance ranging Time-of-Flight sensor based on ST's FlightSense technology.
<https://www.st.com/resource/en/datasheet/vl53l1x.pdf> (accessed April. 03, 2026)
- [3]. Studio 7C - Week 7 - UART and I2C. (accessed April. 03, 2026)
- [4]. I2C, “I2C Programming in TI ARM Tiva”.
<https://labs.dese.iisc.ac.in/embeddedlab/i2c/> (accessed April. 03, 2026)